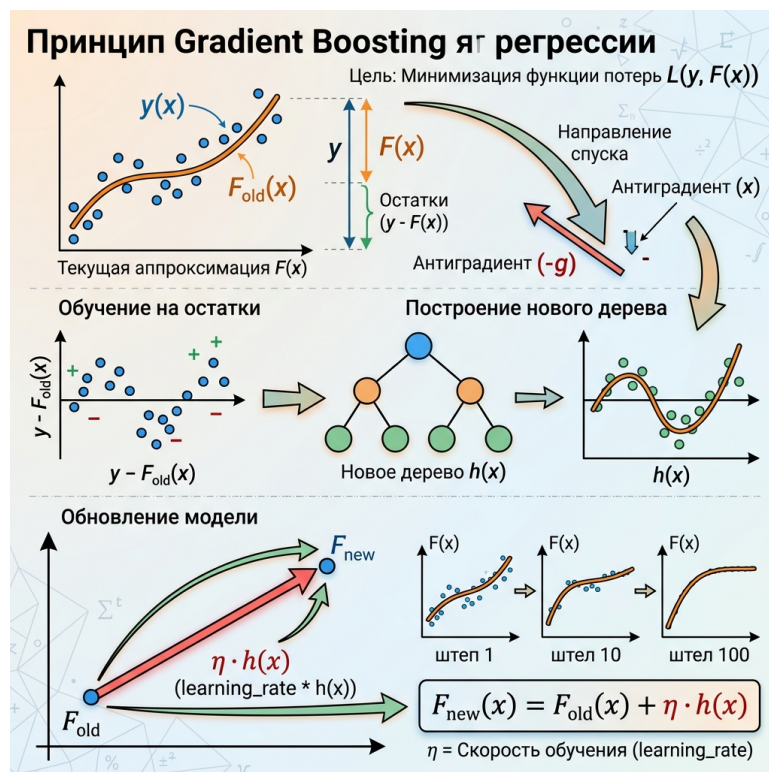


Deep Dive into Machine Learning

Полный справочник для подготовки к собеседованиях



Автор и составитель:
Твой персональный гид Antigravity

4 мая 2026 г.

Введение: Твой путь к Mastery

Этот документ — не просто набор лекций. Это сгусток практического опыта и глубокой теории, структурированный так, чтобы провести тебя от фундаментальных основ до самых мощных алгоритмов современного ML.

Как пользоваться этим справочником:

1. **Фундамент:** Начни с линейных моделей и метрик. Это база, на которой строится всё остальное.
2. **Чистота:** Разделы по кросс-валидации научат тебя не обманывать самого себя при оценке качества.
3. **Алгоритмы:** Переходи к деревьям и бустингу — "тяжелой артиллерии" которая выигрывает соревнования.
4. **Комментарии Antigravity:** Обращай внимание на синие блоки — там собраны инсайты, которые часто спрашивают на собеседованиях.

Удачи в обучении! Помни: в ML нет магии, есть только математика и качественные данные.

Содержание

Введение	1
1 Линейные модели: Фундамент	4
1.1 Типология задач машинного обучения	4
1.2 Математический аппарат и обозначения	4
1.3 Обработка категориальных признаков	4
1.4 Интерпретируемость моделей	4
1.5 Свойства и ограничения линейных моделей	5
1.6 Сведение обучения к задаче оптимизации	5
1.7 Мультиколлинеарность и регуляризация	5
1.8 Сравнение L1 и L2 регуляризаций	5
1.9 Функция потерь перцептрона	5
1.10 Hinge Loss и SVM	5
1.11 Логистическая регрессия и вероятности	5
1.12 Анализ гиперпараметров регуляризации	5
1.13 Softmax для многоклассовой классификации	6
2 Метрики: Как оценивать успех	7
2.1 Типология метрик	7
2.2 Функция потерь vs Метрика качества	7
2.3 Матрица ошибок (Confusion Matrix)	7
2.4 Кривые и площади (AUC)	7
2.5 Усреднение в многоклассовых задачах	7
2.6 Метрики классификации	8
2.7 Метрики регрессии	8
3 Кросс-валидация: Чистота эксперимента	9
3.1 Что такое кросс-валидация?	9
3.2 Метод Hold-out	9
3.3 Стратификация (Stratification)	9
3.4 k-Fold кросс-валидация	10
3.5 Leave-one-out (LOO)	10
3.6 Stratified k-Fold	11
3.7 Кривые обучения (Learning Curves)	11
3.8 Когда стоит заподозрить, что оценка качества завышена?	11
3.9 Примеры «подмешивания» тестовых данных (Data Leakage)	12
4 Метрические методы: k-NN и сходство	14
4.1 Алгоритм k-ближайших соседей (k-NN)	14
4.2 Метрики и функции расстояния	14
4.3 Модификации: Взвешенный k-NN	15
4.4 Ядерная регрессия (Формула Надарая-Ватсона)	15
5 Оптимизация поиска ближайших соседей	15
6 Подбор гиперпараметров: Искусство настройки	17
6.1 Подбор гиперпараметров: определение	17
6.2 Методология подбора	17
6.2.1 Схема Train-Validation-Test	17
6.2.2 Кросс-валидация	17

6.3	Grid Search (Поиск по сетке)	17
6.4	Random Search (Случайный поиск)	18
6.5	Продвинутые методы	18
6.6	Опенсорс-библиотеки	19
7	Решающие деревья: Переход к нелинейности	20
7.1	Фундаментальные свойства решающих деревьев	20
7.2	Жадный алгоритм построения	21
7.3	Критерии ветвления и информативность	21
7.4	Работа с особенностями данных	22
7.5	Регуляризация	22
7.6	Алгоритмические оптимизации (Трюки)	22
8	Градиентный бустинг: Тяжелая артиллерия	23
8.1	Философское отличие: Бустинг vs Бэггинг	23
8.2	Математика: Градиентный спуск в функциональном пространстве	23
8.3	Секрет успеха: Почему деревья неглубокие?	24
8.4	Борьба с «жадностью»: Регуляризация	24
8.5	Тройка лидеров: XGBoost, LightGBM, CatBoost	25
8.6	CatBoost (от Яндекса)	25
8.7	LightGBM (от Microsoft)	25
8.8	XGBoost	25
8.9	Интерпретация: Feature Importance	25

1 Линейные модели: Фундамент

1.1 Типология задач машинного обучения

В обучении с учителем выделяют следующие задачи:

- **Регрессия:** целевая переменная принимает вещественные значения ($Y = \mathbb{R}$).
- **Бинарная классификация:** выбор из двух классов, обычно $\{0, 1\}$ или $\{-1, 1\}$.
- **Многоклассовая классификация:** выбор одного из K классов.
- **Многоклассовая классификация multilabel:** объект может принадлежать нескольким классам одновременно ($Y = \{0, 1\}^K$).
- **Ранжирование:** необходимо отсортировать объекты по их релевантности.

Главное отличие в природе таргетов. В регрессии мы предсказываем число, в классификации — метку класса.

1.2 Математический аппарат и обозначения

- **Матрица X :** матрица «объекты — признаки» размера $N \times D$.
- **Вектор y :** истинные ответы.
- **Вектор весов w :** параметры модели. $y = \langle x, w \rangle + w_0$.
- **Решение МНК:** $w = (X^T X)^{-1} X^T y$.
- **Численные методы:** Для стабильного обращения матриц используют QR-разложение ($X = QR \implies w = R^{-1} Q^T y$) или сингулярное разложение (SVD: $X = U \Sigma V^T \implies w = V \Sigma^{-1} U^T y$).

1.3 Обработка категориальных признаков

One-hot-кодирование заменяет категориальный признак с M значениями на M новых бинарных признаков.

Почему нельзя (1, 2, 3): Линейная модель воспринимает эти числа как шкалу. Она решит, что категория «3» в три раза больше «1», что бессмысленно для профессий или городов.

1.4 Интерпретируемость моделей

Это способность модели объяснить, на основании чего сделано предсказание.

- **Линейные модели:** через веса w_i . Большой положительный вес — признак сильно увеличивает результат.
- **Решающие деревья:** через структуру узлов — путь от корня к листу.

1.5 Свойства и ограничения линейных моделей

- **Узкий класс:** описывают только линейные зависимости. Требуют *feature engineering*.
- **Мультиколлинеарность:** зависимость признаков ведет к огромным и неустойчивым весам.
- **Масштаб:** признаки должны быть отнормированы для правильной интерпретации весов.

1.6 Сведение обучения к задаче оптимизации

- **Функция потерь (Loss):** ошибка на одном объекте.
- **Функционал качества:** средняя ошибка по выборке (Эмпирический риск).
- **SGD (Stochastic Gradient Descent):** Оценка градиента на подвыборке — батче (batch). Обновление весов: $w_{next} = w - \eta \cdot \frac{1}{B} \sum_{t=1}^B \nabla_w L(w, x_{i_t}, y_{i_t})$. Это на порядки быстрее классического градиентного спуска на больших данных.

1.7 Мультиколлинеарность и регуляризация

Регуляризация добавляет штраф за сложность: $\min(Loss + \lambda \|w\|)$. Коэффициент λ контролирует баланс между ошибкой и величиной весов.

1.8 Сравнение L1 и L2 регуляризаций

- L_2 (**Ridge**): $\lambda \sum w_i^2$. Делает веса маленькими, устойчива к шуму.
- L_1 (**Lasso**): $\lambda \sum |w_i|$. Обнуляет веса неважных признаков (разреживание).

1.9 Функция потерь перцептрона

$F(M) = \max(0, -M)$, где $M = y_i \langle w, x_i \rangle$ — отступ. Ошибка растет, если отступ отрицательный.

1.10 Hinge Loss и SVM

$H(M) = \max(0, 1 - M)$. Заставляет модель максимизировать **зазор** (margin). Опорные векторы лежат на границе этого зазора.

1.11 Логистическая регрессия и вероятности

Перевод логита $\langle w, x \rangle$ в вероятность через сигмоиду: $\sigma(z) = \frac{1}{1+e^{-z}}$.

1.12 Анализ гиперпараметров регуляризации

- **Большой λ :** веса стремятся к нулю (недообучение).
- **Маленький λ :** сложная граница, подстройка под выбросы (переобучение).

1.13 Softmax для многоклассовой классификации

$P(y = k|x) = \frac{e^{z_k}}{\sum e^{z_j}}$. Гарантирует, что сумма вероятностей всех классов равна 1.

Antigravity: Линейные модели — это фундамент. Правильно настроенная регуляризация и грамотный фича-инжиниринг делают их мощным инструментом даже в эпоху глубокого обучения.

2 Метрики: Как оценивать успех

2.1 Типология метрик

- **Офлайновые метрики:** вычисляются на исторических данных (Accuracy, ROC-AUC, MSE). Нужны для быстрой проверки гипотез.
- **Онлайновые метрики:** измеряются в реальном времени через А/В-тесты (CTR, выручка).

Модель может иметь идеальный ROC-AUC в офлайне, но «просадить» CTR в онлайне.

2.2 Функция потерь vs Метрика качества

- **Функция потерь (Loss):** инструмент для обучения, **обязательно** дифференцируема.
- **Метрика:** внешний критерий для оценки успеха, может быть любой.

2.3 Матрица ошибок (Confusion Matrix)

	Истинный Pos	Истинный Neg
Предсказали Pos	TP	FP
Предсказали Neg	FN	TN

2.4 Кривые и площади (AUC)

- **ROC-AUC:** Кривая в осях TPR (*Recall*) и FPR ($\frac{FP}{FP+TN}$). Она показывает способность алгоритма верно упорядочивать объекты. AUC равен доле пар объектов разного класса, которые алгоритм верно ранжировал. Не зависит от дисбаланса классов в плане "формы" но может быть обманчив.
- **PR-AUC:** Кривая в осях Precision и Recall. Явно учитывает количество ложных срабатываний и пропусков. Сильно зависит от доли положительного класса, что делает её предпочтительной при сильном дисбалансе.

2.5 Усреднение в многоклассовых задачах

- **Микроусреднение (Micro):** Сначала суммируем элементы матриц ошибок для всех классов, а затем считаем метрику. Важен вклад каждого *объекта*.
- **Макроусреднение (Macro):** Считаем метрику для каждого класса отдельно, а затем усредняем результаты. Важен вклад каждого *класса* (независимо от его размера).

2.6 Метрики классификации

Метрика	Пример задачи	Где ЗАПРЕЩЕНО
Accuracy	Распознавание цифр	Фрод (сильный дисбаланс)
Precision	Таргетинг (высокая цена звонка)	Поиск пропавших (важен каждый сигнал)
Recall	Диагностика рака (нельзя пропустить)	Спам-фильтр (важные письма не в спам)
F1-measure	Поиск в базе знаний	Когда Recall критичнее Precision

2.7 Метрики регрессии

Метрика	Пример задачи	Где ЗАПРЕЩЕНО
MSE	Нагрузка на сервер (штраф за скачки)	При наличии выбросов
MAE	Время доставки (средняя точность)	Когда большие ошибки критичны
MAPE	Выручка (понятно бизнесу в %)	Когда в таргете есть нули
SMAPE	Симметричная % ошибка	Когда важен знак отклонения
WAPE	Взвешенная % ошибка	Когда все объекты равноценны
RMSLE	Прогноз цен (важна относительная разница)	Когда важна абсолютная ошибка в рублях

$$SMAPE: \frac{1}{N} \sum \frac{|\hat{y}_i - y_i|}{(|y_i| + |\hat{y}_i|)/2}, \quad WAPE: \frac{\sum |\hat{y}_i - y_i|}{\sum |y_i|}.$$

Antigravity: Метрики — это мост между бизнесом и математикой. Неверный выбор метрики ведет к созданию идеальной, но бесполезной модели.

Оценка производительности модели

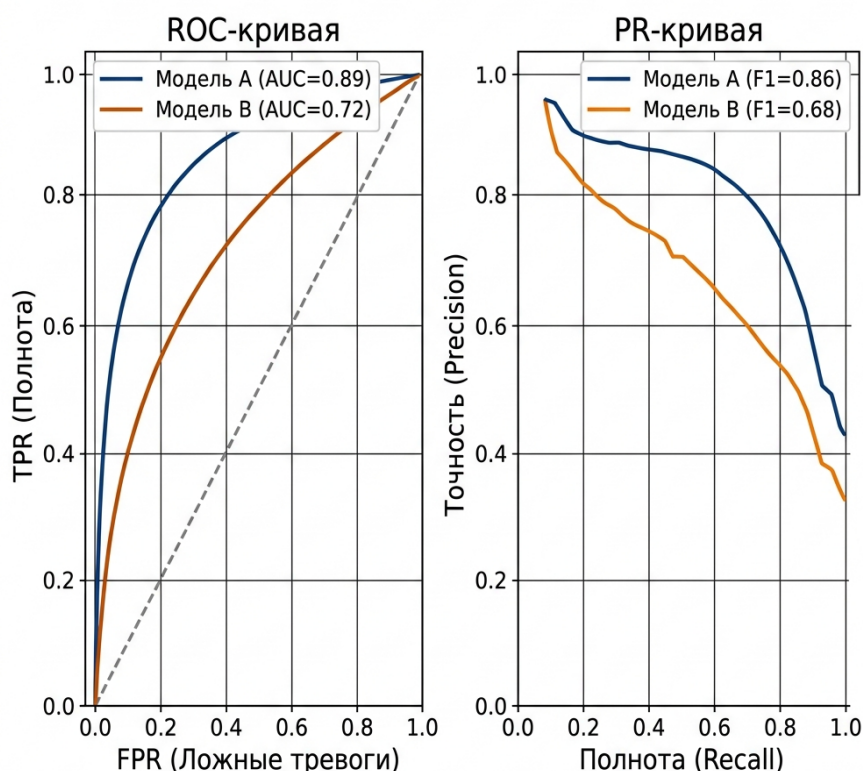


Рис. 1: Сравнение ROC и PR кривых.

3 Кросс-валидация: Чистота эксперимента

3.1 Что такое кросс-валидация?

Кросс-валидация — это процедура оценки качества модели, которая позволяет получить надежную оценку того, как алгоритм будет работать на новых, ранее не виденных данных. Она критически важна для сравнения моделей и подбора гиперпараметров.

3.2 Метод Hold-out

Самый простой и быстрый способ. Мы делим данные на две части:

- **Train (обучающая):** на ней модель учится.
- **Test (тестовая/отложенная):** на ней мы измеряем качество.

Обычно используется пропорция **70/30** или **80/20**.

Минус: Оценка сильно зависит от того, *какие именно* объекты попали в тест. Если в тест случайно попали «легкие» объекты, метрика будет завышена.

3.3 Стратификация (Stratification)

Это техника, гарантирующая, что **распределение классов** (или других важных признаков) в тренировочной и тестовой выборках будет одинаковым.



Рис. 2: Компромисс между точностью и полнотой.

Зачем это нужно? Если у вас в данных 95% здоровых пациентов и 5% больных, при обычном случайном делении в тест могут вообще не попасть больные люди. Модель покажет 100% точность, просто предсказывая «здоров», но на деле она бесполезна.

Решение: Стратификация «нарезает» данные так, чтобы и в трейне, и в тесте сохранялись те же 5% больных.

3.4 k-Fold кросс-валидация

Это «золотой стандарт» оценки.

1. Весь датасет разбивается на k равных частей (фолдов), обычно $k = 5$ или $k = 10$.
2. Модель обучается k раз.
3. На каждой итерации один фолд становится тестовым, а остальные $k - 1$ — тренировочными.
4. Итоговая метрика — это **среднее арифметическое** результатов всех k итераций.

3.5 Leave-one-out (LOO)

Экстремальный случай k-Fold, где k равно количеству объектов в выборке (N).

- **Как это работает:** Мы обучаемся на всех объектах, кроме одного, и на этом одном тестируемся. И так N раз.
- **Плюс:** Мы используем максимум данных для обучения.
- **Минус:** Это невероятно долго и дорого в плане вычислений (если у вас миллион объектов, придется обучить модель миллион раз).

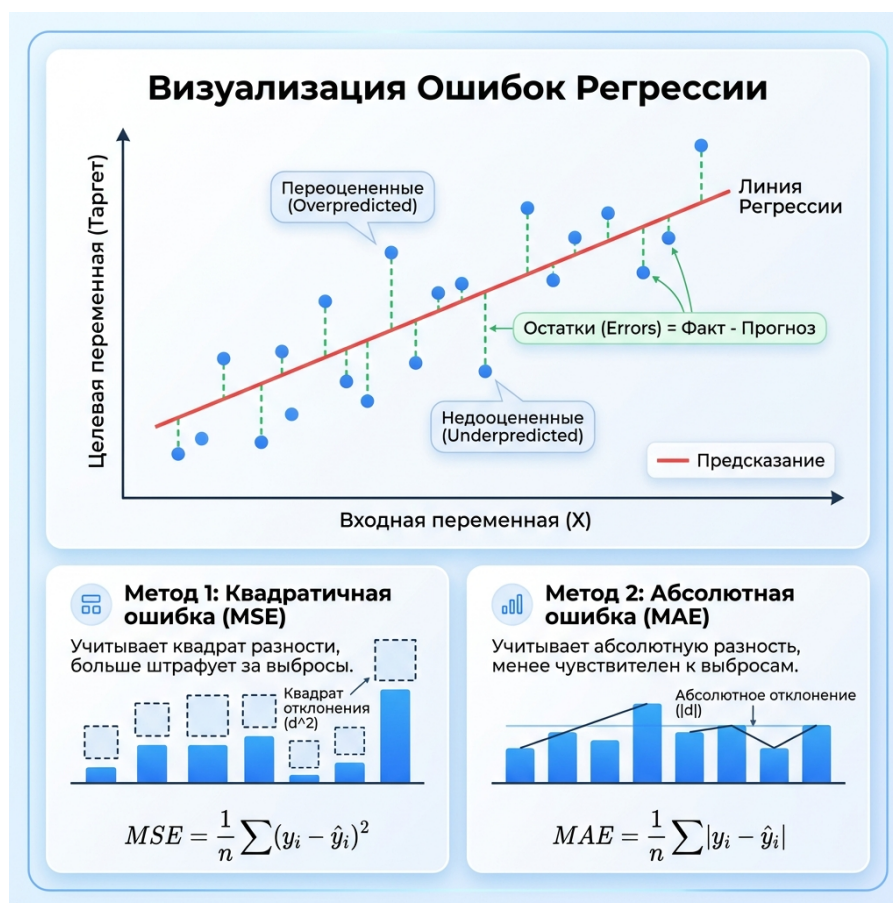


Рис. 3: Влияние выбросов на MSE и MAE.

3.6 Stratified k-Fold

Это комбинация двух методов: данные разбиваются на k фолдов, но при этом в каждом фолде соблюдается **стратификация**. Это самый надежный способ оценки для задач классификации, особенно когда один класс встречается реже другого.

3.7 Кривые обучения (Learning Curves)

Перед внедрением найденную лучшую модель можно обучить на всех имеющихся данных. Правда, вы не сможете оценить качество получившейся модели, так как у вас уже не будет тестового множества. Чтобы примерно оценить, как будет вести себя модель при добавлении новых данных, вы можете построить **кривые обучения**: зависимость качества модели на трейне и на тесте от числа поданных семплов на вход. Это помогает понять, поможет ли сбор дополнительных данных улучшить модель (высокий variance) или модель уперлась в свой предел (высокий bias).

3.8 Когда стоит заподозрить, что оценка качества завышена?

Если вы видите метрику **0.999** или резкий скачок качества по сравнению с бейслайном, не спешите открывать шампанское. Скорее всего, произошел **Overfitting** или **Data Leakage**.

Красные флаги:

1. Метрика на тесте почти такая же идеальная, как на трейне (для сложных задач это редкость).

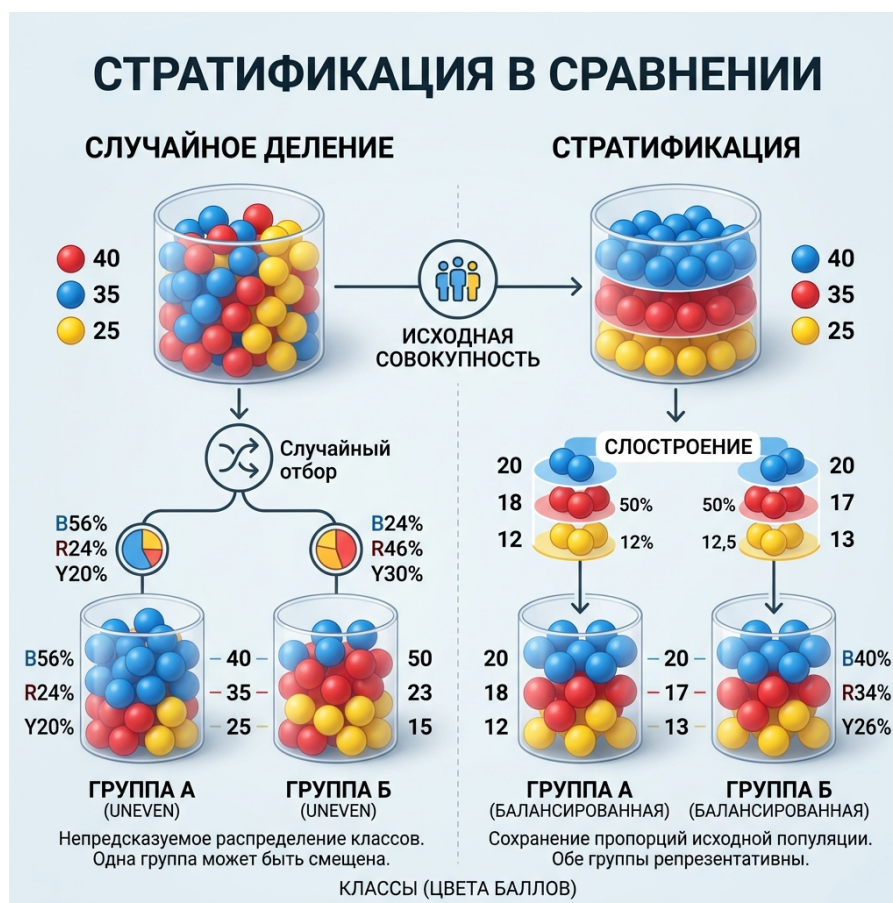


Рис. 4: Визуализация процесса стратификации.

2. Модель отлично работает на кросс-валидации, но «разваливается» на новых реальных данных.
3. В данных есть признак, который подозрительно сильно коррелирует с таргетом (возможно, это таргет в другом облиии).

3.9 Примеры «подмешивания» тестовых данных (Data Leakage)

Это ситуация, когда информация из будущего или из теста просачивается в обучение. Модель «подсматривает» правильные ответы.

Примеры:

- **Масштабирование (Scaling) до разбиения:** Вы посчитали среднее и дисперсию по *всему* датасету, а потом применили `StandardScaler`. Модель уже «знает» среднее значение тестовой выборки. **Правильно:** считать параметры только на трейне.
- **Временные ряды (Time Series):** Вы предсказываете курс акций сегодня, используя данные за завтра. Кросс-валидация моделей для такой задачи осложняется тем, что данные не должны пересекаться по времени: тренировочные данные должны идти до валидационных, а валидационных — до тестовых. С учётом этих особенностей фолды в кросс-валидации для временных рядов располагаются вдоль временной оси (например, метод *Time Series Split* или «скользящее окно»).
- **Дубликаты:** Если в данных есть дублирующие строки, один и тот же объект может попасть и в трейн, и в тест. Модель просто «вспомнит» ответ.

К-КРАТНАЯ КРОСС-ВАЛИДАЦИЯ (5 фолдов)

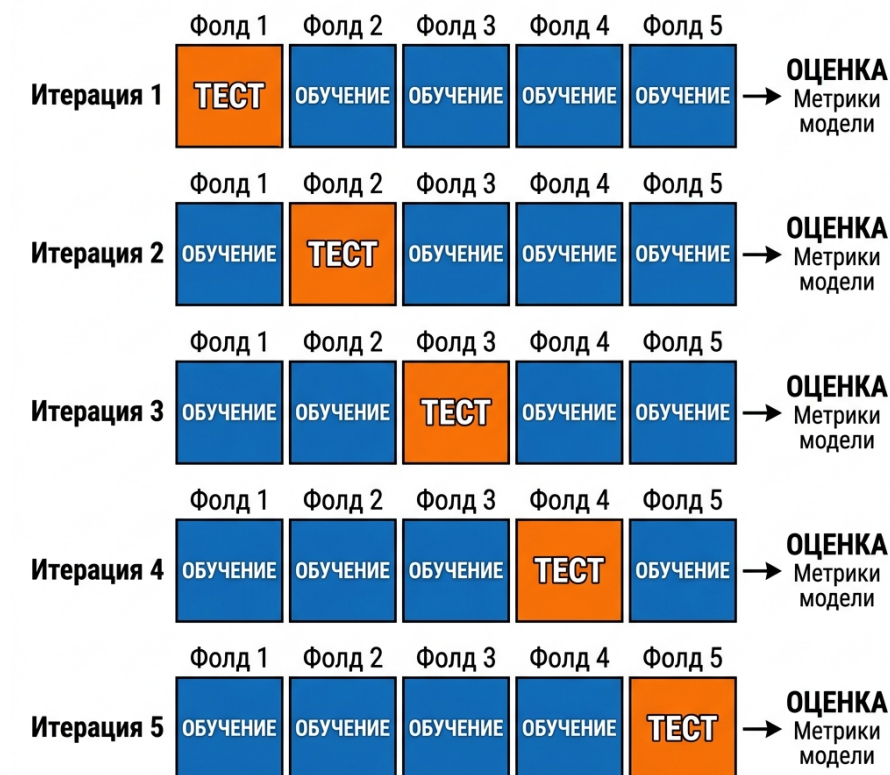


Рис. 5: Схема k-Fold кросс-валидации.

- **Косвенные признаки:** Например, предсказание вероятности рака, используя признак «прошел ли пациент химиотерапию». Терапия назначается *после* диагноза, поэтому она содержит ответ в себе.

Помни, кросс-валидация — это твой страховой полис. Лучше потратить время на обучение 5 фолдов сейчас, чем через месяц объяснять заказчику, почему «идеальная» модель на практике выдает случайные числа.

Antigravity: Всегда используй пайплайны. Это защитит тебя от 90% случаев утечки данных через предобработку.



Рис. 6: Схематичное представление утечки данных.

4 Метрические методы: k-NN и сходство

4.1 Алгоритм k-ближайших соседей (k-NN)

- **Математическая формулировка:** Пусть задана обучающая выборка $X = \{(x_i, y_i)\}$. Для нового объекта x мы находим k ближайших к нему объектов из X : $x_{(1)}, \dots, x_{(k)}$.
- **Классификация:** Ответ — наиболее часто встречающийся класс среди соседей (мода): $a(x) = \operatorname{argmax}_{y \in Y} \sum_{j=1}^k [y_{(j)} = y]$.
- **Регрессия:** Ответ — среднее арифметическое ответов соседей: $a(x) = \frac{1}{k} \sum_{j=1}^k y_{(j)}$.
- **Параметр k :**
 - При малых k модель имеет **низкое смещение (bias)**, но **высокий разброс (variance)**.
 - При больших k модель имеет **высокое смещение**, но **низкий разброс**.

Выбор k — это классический поиск компромисса между недообучением и переобучением. Обычно k подбирают на кросс-валидации.

4.2 Метрики и функции расстояния

- **Метрики Минковского (L_p):** $\rho(x, z) = \left(\sum_{i=1}^d |x_i - z_i|^p \right)^{1/p}$.
 - L_1 (Манхэттенское): сумма модулей разностей. Менее чувствительна к выбросам.
 - L_2 (Евклидово): стандартное расстояние по прямой.

- **Косинусное сходство:** $\cos(\theta) = \frac{\langle x, z \rangle}{\|x\| \|z\|}$. Расстояние определяется как $1 - \cos(\theta)$. Измеряет угол между векторами, игнорируя их длину.
- **Расстояние Жаккара:** $1 - \frac{|A \cap B|}{|A \cup B|}$. Используется для сравнения множеств (стандарт для бинарных признаков).

*Помните, что перед использованием L_p метрик данные **обязательно** нужно масштабировать, иначе признак с большим диапазоном значений «поглотит» все остальные.*

4.3 Модификации: Взвешенный k-NN

- **Веса по рангу:** Вес j -го соседа зависит от его номера: $w_j = \frac{k+1-j}{k}$.
- **Веса от расстояния:** $w_i = f(\rho(x, x_i))$. Позволяет доверять более близким объектам сильнее.
- **Ядерное сглаживание:** Вес задается через ядро K : $w_i = K\left(\frac{\rho(x, x_i)}{h}\right)$.
- **Примеры ядерных функций** (в порядке увеличения их гладкости):
 - Прямоугольное: $K(z) = \frac{1}{2} [|z| \leq 1]$.
 - Треугольное: $K(z) = (1 - |z|) [|z| \leq 1]$.
 - Епанечникова: $K(z) = \frac{3}{4} (1 - z^2) [|z| \leq 1]$.
 - Гауссовское: $K(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$ (бесконечно гладкое).

4.4 Ядерная регрессия (Формула Надарая-Ватсона)

- **Принцип:** Это обобщение k-NN, где предсказание — это взвешенное среднее ответов всех объектов обучающей выборки: $a(x) = \frac{\sum_{i=1}^n y_i K\left(\frac{\rho(x, x_i)}{h}\right)}{\sum_{i=1}^n K\left(\frac{\rho(x, x_i)}{h}\right)}$.
- **Ширина окна h :**
 - Малое h : веса быстро затухают (риск переобучения).
 - Большое h : прогноз становится слишком гладким (риск недообучения).

5 Оптимизация поиска ближайших соседей

- **k-d деревья:** Бинарное дерево, разбивающее пространство вдоль осей. Эффективно при $d < 20$.
- **Приближенные методы (ANN):** Согласие найти "достаточно близкого" соседа для ускорения в тысячи раз.
- **Locality-sensitive hashing (LSH):** Хеш-функция переводит близкие объекты в один бакет. Вероятность коллизии для близких объектов должна быть высокой, а для далёких — низкой.
- **Hierarchical Navigable Small World (HNSW):** На данных строится многоуровневый NSW-граф, где между любыми двумя точками существует короткий путь. Исходный граф — нулевой слой, а каждый следующий содержит всё меньше вершин, позволяя быстро "перепрыгивать" по пространству.

Antigravity: В реальном продакшене (Faiss, HNSW) почти всегда используются именно ANN, так как точный поиск на миллионах объектов невозможен.

6 Подбор гиперпараметров: Искусство настройки

6.1 Подбор гиперпараметров: определение

В машинном обучении различают два типа переменных:

- **Параметры модели:** это веса, которые алгоритм находит самостоятельно в процессе обучения на данных (например, коэффициенты w в линейной регрессии).
- **Гиперпараметры:** это внешние настройки алгоритма, которые задаются исследователем до начала обучения. Они определяют саму структуру модели и способ её обучения (например, глубина дерева, количество соседей в KNN, сила регуляризации).

Подбор гиперпараметров — это процесс поиска таких значений, при которых модель показывает наилучшее качество на новых данных.

6.2 Методология подбора

Чтобы оценка качества была честной, нельзя подбирать гиперпараметры на тех же данных, на которых модель обучается или на которых она окончательно тестируется.

6.2.1 Схема Train-Validation-Test

Выборка делится на три части:

1. **Train (Обучающая):** для настройки внутренних параметров (весов).
2. **Validation (Валидационная):** для оценки качества при разных гиперпараметрах и выбора лучшего набора.
3. **Test (Тестовая):** для финальной проверки качества. К этой части обращаются один раз в самом конце.

6.2.2 Кросс-валидация

Если данных мало, используется **k-Fold кросс-валидация**. Весь тренировочный сет делится на k частей. Модель обучается k раз, каждый раз используя новую часть как валидационную. Это позволяет более эффективно использовать данные.

6.3 Grid Search (Поиск по сетке)

Принцип: Исследователь задает список значений для каждого гиперпараметра. Алгоритм перебирает **все возможные комбинации**.

- **Плюсы:** Гарантированно находит лучший вариант из предложенной сетки.
- **Минусы:** «Проклятие размерности». Если параметров много, количество комбинаций растёт экспоненциально.



Рис. 7: Схема разделения данных для честной оценки.

6.4 Random Search (Случайный поиск)

Принцип: Значения выбираются случайно из заданных распределений.

- **Плюсы:** Вероятность того, что хотя бы один насэмплированный набор попал в лучшую группу (топ-5%), равна $1 - (1 - 0,05)^n$. При $n \geq 60$ мы попадём в этот топ с вероятностью не менее 0,95. Это значительно быстрее, чем перебор всех комбинаций в Grid Search. Эффективнее при большом количестве параметров, так как тратит время только на важные измерения.
- **Минусы:** Может пропустить «идеальный» узкий пик качества.

6.5 Продвинутые методы

- **Байесовская оптимизация:** Строит вероятностную (суррогатную) модель функции потерь. Для выбора следующей точки используется **acquisition function** (функция полезности), которая балансирует между *exploration* (исследование точек с большой дисперсией модели) и *exploitation* (исследование точек с высоким ожидаемым качеством). Пример: $\alpha(x) = \mu(x) + \beta\sigma(x)$.
- **Population Based Training (PBT):** Метод, объединяющий эволюционные алгоритмы и параллельное обучение. Процедура **exploit()**: если у модели низкое каче-



Рис. 8: Сравнение Grid Search и Random Search.

ство, её веса заменяются весами лучшей модели популяции. Процедура `explore()`: добавление случайного шума в гиперпараметры после перезаписи весов.

- **Алгоритм TPE (Tree-structured Parzen Estimator):** Продвинутый вариант байесовской оптимизации, используемый в Hyperopt и Optuna.

6.6 Опенсорс-библиотеки

Библиотека	Особенности
Scikit-learn	Базовые GridSearchCV и RandomizedSearchCV.
Hyperopt	Классика для метода TPE.
Optuna	Самая современная: define-by-run, быстрая, отличная визуализация.
skopt	Байесовская оптимизация с интерфейсом sklearn.
Keras Tuner	Инструмент для подбора архитектур нейросетей.

Antigravity: Начинай всегда с Random Search, чтобы быстро прошупать почву. Если задача критически важна — переходи на Optuna.

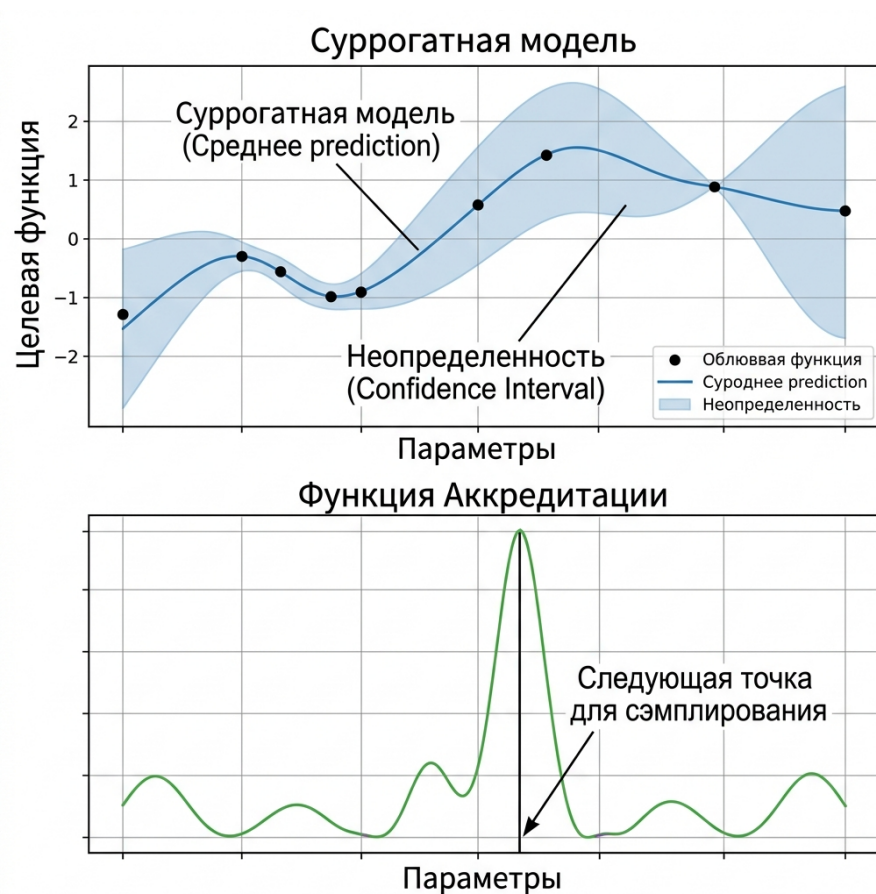


Рис. 9: Интуиция байесовской оптимизации.

7 Решающие деревья: Переход к нелинейности

7.1 Фундаментальные свойства решающих деревьев

- **Вопрос:** Почему функция дерева является кусочно-постоянной и как это влияет на оптимизацию?
- **Ответ:** Решающее дерево разбивает признаковое пространство на непересекающиеся гиперпрямоугольники (листья). В каждом листе модель предсказывает одно и то же значение (среднее или моду). Математически это сумма индикаторных функций: $a(x) = \sum_{j=1}^J c_j [x \in L_j]$. Так как эта функция разрывна, а её производная по параметрам разбиения (порогам) почти везде равна нулю или не существует, **градиентные методы** нельзя использовать для построения структуры самого дерева.
- **Вопрос:** Почему дерево не умеет экстраполировать?
- **Ответ:** В отличие от линейных моделей, которые могут бесконечно продолжать прямую линию, дерево «заперто» в значениях обучающей выборки. Если новый объект имеет значения признаков далеко за пределами тренировочных, он все равно попадет в один из крайних листьев и получит константное предсказание этого листа.

Дерево — это отличный интерполятор, но плохой предсказатель «будущих рекордов». Оно никогда не предскажет цену квартиры выше, чем та, что была в обучении.

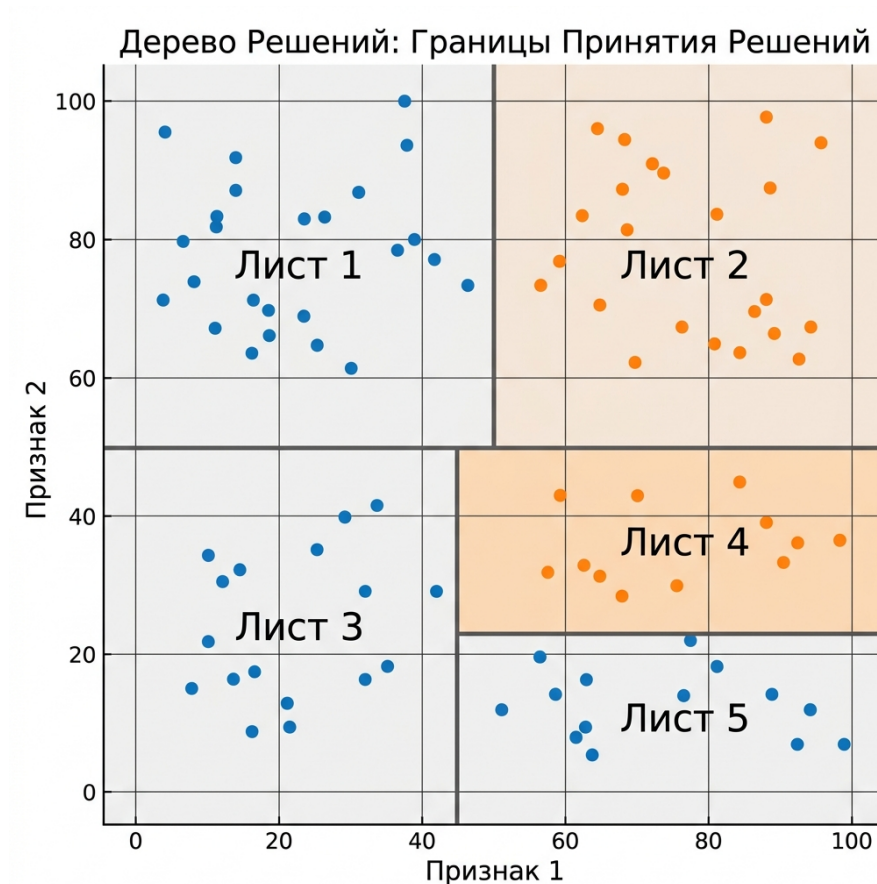


Рис. 10: Разбиение признакового пространства на гиперпрямоугольники (листы).

7.2 Жадный алгоритм построения

- **Вопрос:** Почему построение оптимального дерева — NP-трудная задача и в чем суть «жадности»?
- **Ответ:** Количество возможных структур дерева растет комбинаторно. Чтобы найти глобально оптимальное дерево, нужно было бы перебрать все возможные последовательности разбиений, что невозможно. **Жадный алгоритм** на каждом шаге ищет один лучший сплит (признак и порог), который максимально уменьшает ошибку прямо сейчас, не заглядывая в будущее.

Жадность — это как выбор самого вкусного куска торта прямо сейчас вместо того, чтобы сесть на диету ради здоровья в будущем. Это работает быстро, но не гарантирует идеальный результат для всей структуры.

7.3 Критерии ветвления и информативность

- **Gain (Прирост информативности):** $G = I(\text{parent}) - \frac{N_{\text{left}}}{N} I(\text{left}) - \frac{N_{\text{right}}}{N} I(\text{right})$. Мы хотим, чтобы неопределенность (I) в дочерних вершинах была минимальной.
- **MSE (для регрессии):** Это просто дисперсия таргетов в узле. Минимизируя её, мы делаем объекты в листе максимально похожими на их среднее.
- **Энтропия Шеннона:** $H(X_m) = -\sum_{k=1}^K p_k \log p_k$. Она измеряет непредсказуемость реализации случайной величины. Наибольшего значения энтропия достигает для

равномерно распределённой случайной величины (максимальный хаос), а минимального (нуля) — когда в узле только один класс.

- **Критерий Джини:** $H(X_m) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum p_k^2$. Он равен математическому ожиданию числа неправильно классифицированных объектов в случае, если мы будем приписывать им случайные метки из дискретного распределения классов в данном узле.

Критерий доли ошибок плох тем, что он «грубый» — он может не измениться после сплита, даже если классы стали более разделёнными.

7.4 Работа с особенностями данных

- **Категории:** Помимо ONE, в деревьях можно использовать группировку категорий. Категории сортируют по среднему значению таргета и ищут порог так же, как в числовом признаке.
- **Пропуски:**
 1. Можно отправлять пропущенные значения в ту сторону, где ошибка меньше.
 2. **Суррогатные сплиты:** Если основной признак пропущен, дерево использует «запасной» признак, который сильнее всего коррелирует с основным.

7.5 Регуляризация

- **Max depth:** Глубина дерева.
- **Min samples leaf:** Минимум объектов в листе.
- **Max leaf nodes:** Ограничение на количество ответов модели.
- **Min impurity decrease:** Не делать сплит, если улучшение незначительно.

7.6 Алгоритмические оптимизации (Трюки)

- **Сортировка ($O(N \log N)$):** Позволяет обновлять статистики за $O(1)$ при переборе порогов.
- **Гистограммный метод:** Отрезок между максимальным и минимальным значением признака разбивается на b подотрезков (бинов), длины которых выбираются так, чтобы в каждый попадало примерно равное число обучающих примеров. После чего значения признака заменяются на номера бинов. Это на порядки ускоряет обучение и экономит память, так как вместо перебора N уникальных значений мы перебираем лишь b границ.

Antigravity: Гистограммный метод — это "секретный соус" современных библиотек (LightGBM, CatBoost), позволяющий обучать деревья на миллионах строк за секунды.

8 Градиентный бустинг: Тяжелая артиллерия

8.1 Философское отличие: Бустинг vs Бэггинг

Чтобы понять бустинг, нужно вспомнить случайный лес (бэггинг).

- **Бэггинг** (Random Forest) строит много глубоких, умных деревьев **независимо** и усредняет их. Это борется с **разбросом** (variance).
- **Бустинг** строит цепочку из очень простых («глупых») деревьев **последовательно**. Каждое следующее дерево исправляет ошибки всех предыдущих. Это борется со **смещением** (bias).

Бустинг можно представить как гольфиста, цель которого — загнать мяч в лунку. Здесь положение мяча — ответ композиции. Каждый следующий удар — это та поправка, которую вносит очередной базовый алгоритм. Если гольфист всё делает правильно, то функция потерь будет уменьшаться с каждым ударом.

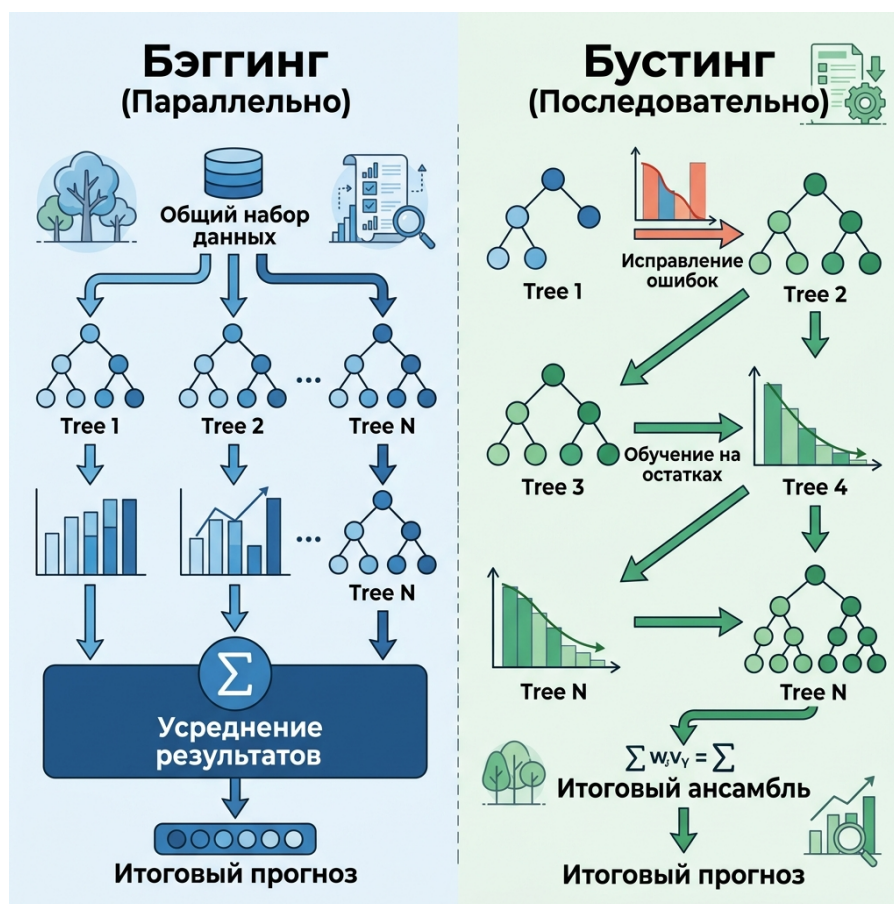


Рис. 11: Концептуальное различие между Бустингом и Бэггингом.

8.2 Математика: Градиентный спуск в функциональном пространстве

В градиентном бустинге мы меняем **саму функцию** $a(x)$.

Представь, что текущая композиция a_{k-1} выдает предсказания. Мы хотим добавить к ней дерево b_k , чтобы общая ошибка упала.

1. **Градиент:** $g_i = \left. \frac{\partial \mathcal{L}(y_i, z)}{\partial z} \right|_{z=a_{k-1}(x_i)}$. Это число говорит, куда нужно «подвинуть» предсказание для уменьшения ошибки.
2. **Антиградиент:** Для наискорейшего уменьшения функции потерь нам необходимо шагнуть в сторону её антиградиента по вектору предсказаний текущей композиции, то есть в сторону вектора $-g$.
3. **Универсальность:** Для каждого объекта очередной алгоритм b_k в бустинге обучается предсказывать антиградиент функции потерь по предсказанию модели. Это наблюдение позволяет обобщить подход построения бустинга на **произвольную дифференцируемую функцию потерь**.

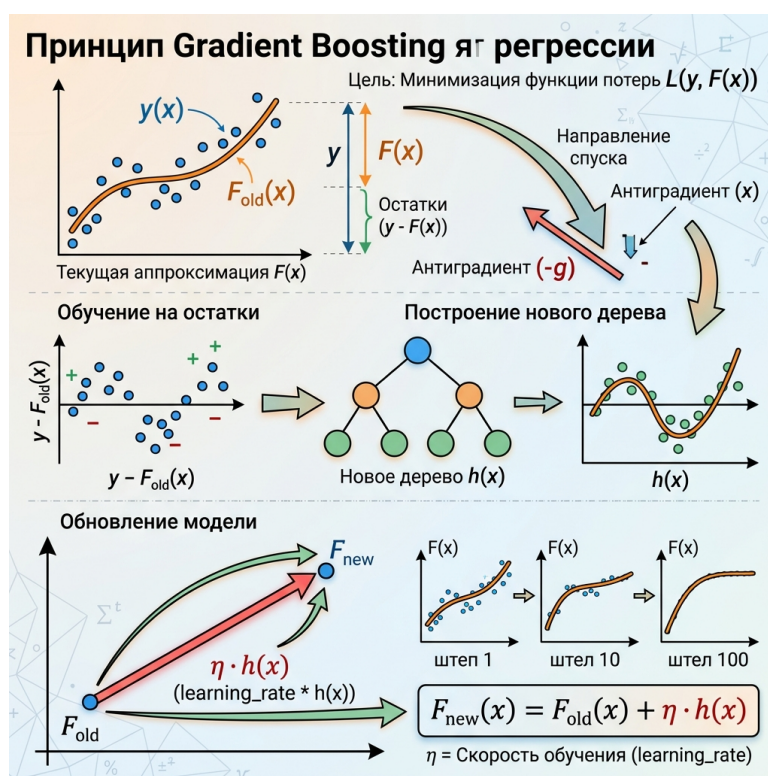


Рис. 12: Процесс обучения нового дерева на антиградиент функции потерь.

8.3 Секрет успеха: Почему деревья неглубокие?

В GBDT базовые деревья обычно имеют глубину всего **3–6 уровней**. Смысл бустинга в том, чтобы каждое дерево вносило **крошечный, но точный вклад**. Тысяча маленьких уточнений работают лучше, чем пять больших и грубых.

8.4 Борьба с «жадностью»: Регуляризация

- **Learning Rate (Shrinkage):** Мы берем лишь малую часть предсказаний нового дерева (например, 0.1). Это делает алгоритм устойчивым к шуму.
- **Стохастичность:** Использование случайных подмножеств данных для построения каждого дерева (Stochastic Gradient Boosting).

Сравнение структур деревьев



Рис. 13: Различия в структуре деревьев: CatBoost (симметричные) vs LightGBM (полистовые).

8.5 Тройка лидеров: XGBoost, LightGBM, CatBoost

8.6 CatBoost (от Яндекса)

- **Симметричные деревья:** Все узлы на одном уровне используют один и тот же признак.
- **Категориальные признаки:** Продвинутое кодирование "из коробки".

8.7 LightGBM (от Microsoft)

- **Leaf-wise рост:** Выбирает самый «перспективный» лист и разбивает его.
- **Гистограммный метод:** Сжатие признаков в «корзины» для ускорения.

8.8 XGBoost

- **Вторая производная:** Использует разложение Тейлора до второго порядка (Гесс-сиан) для точности.

8.9 Интерпретация: Feature Importance

1. **Weight (Frequency):** Частота использования признака.
2. **Gain:** Суммарное падение ошибки от признака.

3. **Permutation Importance:** Падение качества при перемешивании значений.

*Главное при работе с бустингом — следить за графиком лосса на валидации. Как только ошибка на валидации перестала падать, а на трейне продолжает — немедленно останавливайся (это называется *Early Stopping*).*

Antigravity: Для большинства табличных задач CatBoost — это выбор по умолчанию благодаря отличной работе с категориями и регуляризации.